OpenResearch Project Documentation

Last updated: January 17, 2026

1) Overview

OpenResearch is a collaboration-first research platform that combines real-time teamwork with AI-assisted research workflows. It provides group chat, paper discovery, group-level paper libraries, AI Q&A and summarization, and PDF reporting—built with a modern full-stack architecture.

Core goals

- Enable research teams to collaborate in real time.
- Provide AI features that are intentional and auditable via the `@ai` trigger.
- Support semantic search across group papers using vector embeddings.

2) System Architecture

The platform is organized as four primary services:

1. **Frontend (Next.js 16)**
   - App Router, React 19, Tailwind CSS
   - Provides all UI pages: auth, groups, sessions, papers, reports
2. **Backend API (Node.js 20 + Express)**
   - REST API, JWT auth, Socket.IO for real-time events
   - Acts as the primary API for the client
3. **AI Service (FastAPI + Python 3.12)**
   - Handles embeddings, contextual AI Q&A, summarization, and report generation
   - Communicates with Gemini and PostgreSQL
4. **Database (PostgreSQL 16 + pgvector)**
   - Stores app data and vector embeddings (HNSW index for fast similarity search)

3) Repository Layout

```
OpenResearch/
├── client/              # Next.js frontend
├── server/              # Node.js API + Socket.IO
├── ai-service/          # FastAPI AI service
├── docs/                # Documentation
├── scripts/             # Utility scripts
├── docker-compose.yml   # Docker stack
└── README.md            # Main overview
```

4) Services in Detail

4.1 Frontend (client/)

- **Framework:** Next.js 16 + React 19
- **Key responsibilities:** UI/UX, authentication flow, group management, chat, paper discovery, AI interactions
- **Real-time:** Socket.IO client integration for chat & typing indicators

Important directories

- `app/`: App Router pages
- `components/`: UI and layout components
- `lib/`: API client, auth state, socket hook

4.2 Backend API (server/)

- **Framework:** Express 5
- **Auth:** JWT access + refresh tokens
- **Database:** Drizzle ORM (PostgreSQL)
- **Real-time:** Socket.IO server
- **AI integration:** Proxies to the FastAPI service

Key routes

- `/api/auth/*` – authentication
- `/api/groups/*` – group management
- `/api/sessions/*` – chat sessions and messages
- `/api/papers/*` – paper search and saved papers
- `/api/ai/*` – AI requests (proxied)

4.3 AI Service (ai-service/)

- **Framework:** FastAPI
- **AI model:** Google Gemini 2.0 Flash
- **Capabilities:**
  - Q&A with session + paper context
  - Session summarization
  - Embeddings + vector search (pgvector)
  - PDF report generation

Primary endpoints

- `GET /health`
- `POST /test`
- `POST /chat`
- `POST /summarize`

4.4 Database (PostgreSQL + pgvector)

Core tables

- `users`, `groups`, `group_members`
- `sessions`, `messages`

- papers, saved_papers, session_papers
- refresh_tokens
- vector-related tables for embeddings and group context

5) Development Setup (Local)

5.1 Prerequisites

- Node.js 20+
- Python 3.12+
- PostgreSQL 16+

5.2 Backend API

```
cd server
npm install
cp .env.example .env
# edit .env
npm run db:push
npm run db:seed
npm run dev
```

5.3 Frontend Client

```
cd client
npm install
cp .env.example .env.local
# edit .env.local
npm run dev
```

5.4 AI Service

```
cd ai-service
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# edit .env
uvicorn app.main:app --reload --port 8000
```

5.5 Local URLs

- Frontend: http://localhost:3000
- API: http://localhost:3001
- AI Service: http://localhost:8000

- AI Docs: http://localhost:8000/docs

6) Docker (All-in-One)

A full-stack Docker setup is provided in docker-compose.yml.

```
# From repo root
docker compose up --build
```

Services exposed:

- `client` → 3000
- `server` → 3001
- `ai-service` → 8000
- `postgres` → 5432

7) Environment Variables

7.1 Server (.env)

Variable	Description	Required
`DATABASE_URL`	PostgreSQL connection string	Yes
`JWT_SECRET`	JWT signing secret	Yes
`JWT_REFRESH_SECRET`	Refresh token secret	Yes
`PORT`	Server port (default 3001)	No
`CLIENT_URL`	Frontend URL for CORS	No
`NODE_ENV`	development/production	No

7.2 Client (.env.local)

Variable	Description	Required
`NEXT_PUBLIC_API_URL`	Backend API URL	Yes
`NEXT_PUBLIC_WS_URL`	WebSocket URL	Yes
`NEXT_PUBLIC_AI_URL`	AI service URL (optional in UI)	No

7.3 AI Service (.env)

Variable	Description	Required
`GEMINI_API_KEY`	Google Gemini API key	Yes
`DATABASE_URL`	PostgreSQL connection string	Yes

Variable	Description	Required
GEMINI_MODEL	Model name (default gemini-2.0-flash-exp)	No
DEBUG	Verbose logging	No
MAX_CONTEXT_MESSAGES	Max messages in context	No
MAX_CONTEXT_TOKENS	Max tokens in context	No
REQUEST_TIMEOUT	Gemini timeout (seconds)	No

8) AI Usage & @ai Trigger

All AI features require the `@ai` trigger in chat to avoid accidental calls and to make usage intentional and auditable.

Examples

- `@ai Summarize our last session`
- `@ai What methodology is used in this paper?`

9) Testing

Backend (server/)

```
npm test
npm run test:coverage
```

AI Service (ai-service/)

```
python -m pytest
```

Client (client/)

```
npm test
```

10) Typical Workflows

10.1 Create a Group

1. Sign in
2. Create a new group
3. Invite members by email

10.2 Start a Session

1. Open a group
2. Create a new session
3. Start chatting in real time

10.3 Paper Discovery & Saving

1. Search arXiv from the Papers view
2. Save papers to the group library
3. Link papers to sessions

10.4 AI Q&A

1. Open a session chat
2. Use `@ai` with a question
3. The AI uses session messages and linked papers

10.5 Reports

1. Generate a report from the Reports view
2. Download as PDF

11) Troubleshooting

Issue: AI service is unreachable

- Ensure FastAPI is running on port 8000
- Verify `AI_SERVICE_URL` in server environment

Issue: 401 Unauthorized

- Check JWT secrets
- Confirm access token refresh flow

Issue: Database connection errors

- Verify `DATABASE_URL`
- Ensure PostgreSQL is running

Issue: Socket.IO not connecting

- Verify `NEXT_PUBLIC_WS_URL`
- Check CORS and server port

12) Security Notes

- JWT secrets must be long and random (32+ characters).
- AI access is explicit via `@ai` to control costs and audit usage.
- Use HTTPS in production and set strict CORS origins.

13) Contribution Guidelines

- Keep changes scoped and documented.

- Add or update tests for new behavior.
- Follow existing code style and linting rules.

14) Related Docs

- AI features: docs/features/ai-features.md
- Group context & RAG: docs/features/group-context.md
- Root overview: README.md